SYSTEM DESIGN INTERVIEW | FREE PRINTABLE GUIDE

30 questions and answers

15 interactive design prompts with answer outlines and worked walkthroughs, plus 15 system design fundamentals with direct answers and links to deeper guides. Free to download, with no account or email required.

Use each outline as a starting point. In an interview, clarify requirements and scale first; a sound design depends on the constraints you agree on.

Design a URL Shortener

Answer outline. At 10 billion redirects a month, plan for about 3,850 average and 19,000 peak redirects per second. Generate random short codes with a Postgres primary key and collision retries, serve redirects through a CDN/cache, and count clicks asynchronously. Short TTLs plus purges make takedowns fast.

Practice focus. Estimate a read-heavy workload, then decide what to cache and how to count clicks without slowing redirects.

Read the full worked answer

Design an API Rate Limiter

Answer outline. Use a token bucket per API key, with the plan's rate and burst size stored alongside the counter. Run each refill and consume atomically in Redis so requests across API servers share one limit. Return 429 with Retry-After when empty. If Redis is unavailable, apply approximate local limits so the API stays available.

Practice focus. Compare counter and token-bucket designs, then decide how multiple servers share limits.

Read the full worked answer

Design a News Feed (Twitter Timeline)

Answer outline. Store each post once, then fan its ID out asynchronously to the recent timelines of active followers. Skip write-time fan-out for accounts with tens of millions of followers and merge their recent posts when a reader opens the feed. Keep timelines as bounded, rebuildable lists of IDs; hydrate and filter posts on the read path.

Practice focus. Choose between write fan-out and read fan-out when a few accounts have millions of followers.

Read the full worked answer

Design an Uber-Style Ride-Sharing System

Answer outline. Store each ride request and driver reservation durably, then enqueue matching work through an outbox. Use a spatial cell index to find nearby drivers with fresh location pings, rank a small candidate set by road ETA, and issue expiring offers. Conditional state changes ensure one driver and one ride can be assigned only once, even when replies arrive late or twice.

Practice focus. Design Uber-style dispatch: match nearby drivers quickly without stale locations or double assignments, then isolate a stadium hotspot.

Read the full worked answer

Design a Payment System

Answer outline. Create one payment attempt per order and call the provider with a stable idempotency key. Treat a timeout as unknown, then look up the provider's result instead of issuing a new charge. Record signed, duplicate or out-of-order webhooks in an append-only event ledger, apply only valid state transitions, and use an outbox for fulfillment and receipts.

Practice focus. Make retries safe when a payment succeeds but the caller never receives the response.

Read the full worked answer

Design a Distributed Job Queue

Answer outline. Put a durable Kafka log in front of the existing Redis worker queues. A relay retries delivery until Redis accepts each job, while workers lease jobs and acknowledge only after success, so a crash can repeat work but cannot lose it. Separate queues by job type, throttle each against its dependencies, and drain backlogs at a rate downstream systems can handle.

Practice focus. Handle duplicate delivery, worker crashes, retries and messages that can never succeed.

Read the full worked answer

Design a Video Processing Pipeline

Answer outline. Let browsers upload video directly to object storage with resumable multipart uploads. A durable job record and queue hand each video to a worker under a lease; fencing prevents an expired worker from publishing stale output. Verify every rendition, then atomically mark the video ready. Retries may repeat work, but they cannot expose partial or duplicate results.

Practice focus. Separate upload, durable job state and transcoding so each can recover from failures independently.

Read the full worked answer

Design a Dropbox-Style File Sync Service

Answer outline. Split files into verified immutable blocks and publish each version through a conditional metadata manifest. Devices resume uploads by sending only missing blocks, then catch up from a durable per-account change journal. Keep concurrent offline edits as separate versions when they cannot be merged, and check current permissions whenever a device reads shared content.

Practice focus. Keep large uploads resumable and file versions complete while devices reconnect, conflict, lose access and restore deleted data.

Read the full worked answer

Design a Product Analytics System

Answer outline. A capture API validates batches and appends events with stable IDs to a durable, partitioned stream. Workers resolve identities and insert large batches into a column store ordered for team and time-range queries. Keep rare properties in JSON, promote frequently filtered properties to typed columns, and deduplicate retries by stream partition and offset.

Practice focus. Keep ingestion durable and deduplicated while making time-range analytics fast for each customer.

Read the full worked answer

Design a Notification System

Answer outline. Write product events to an outbox, then let a planner create one deduplicated delivery per user and channel. Put critical, transactional and bulk work on separate queues, and apply preferences again just before sending so unsubscribes take effect. Provider outages delay queued notifications without blocking the product action that caused them.

Practice focus. Prevent duplicate sends, honor preference changes and protect urgent messages when a provider slows down.

Read the full worked answer

Design a Distributed Cache (Memcache)

Answer outline. Use a look-aside cache in front of the database and delete cache entries after writes. Give one reader a short-lived lease to refill a missing key, which prevents concurrent misses from stampeding the database and stops an older read from restoring stale data after a delete. Rehash around failed cache nodes and propagate invalidations across clusters.

Practice focus. Control stale reads and database load during writes, hot-key bursts and cache failures.

Read the full worked answer

Design Discord's Message Storage

Answer outline. Partition messages by channel and ten-day time bucket, then sort each partition by a time-ordered message ID. Recent-page reads stay small, while old-message lookups can target the right bucket. Keep tombstones short-lived, repair partial rows, and use verified dual writes to move the dataset while it stays online.

Practice focus. Make channel history cheap to read while keeping hot channels and database migrations manageable.

Read the full worked answer

Design a Collaborative Editor (Google Docs)

Answer outline. Route every document to one owner that sequences edits and appends them to a durable operation log before acknowledging them. Clients apply local edits immediately, then reconcile concurrent operations with a defined OT or CRDT algorithm; reconnecting clients replay missing operations by sequence. Keep cursors and presence as short-lived state outside the durable edit log.

Practice focus. Preserve one document history through concurrent edits, reconnects, owner changes and fan-out.

Read the full worked answer

Design a Reactive Database (Live Queries)

Answer outline. Run each query against a versioned snapshot and record the index ranges it reads. After a commit, compare its writes with those read sets and rerun only affected subscriptions; tag results with one timestamp so a screen never mixes database moments. Validate mutations against the same read sets to preserve rules under concurrent writes.

Practice focus. Update subscribed query results without polling or showing data from mixed snapshots.

Read the full worked answer

Shard a Live Database Without Downtime

Answer outline. Shard by workspace because common reads and joins stay within one workspace, and map 480 logical Postgres shards to hosts. Log writes, backfill versioned rows, verify with comparisons and dark reads, then cut over in a short maintenance window. Replicas and table partitioning address other limits, but neither divides one host's write load.

Practice focus. Scale write volume without rehashing every row, and migrate live data without losing newer writes.

Read the full worked answer

15 fundamentals, with direct answers

Use these concise explanations to review common trade-offs, failure handling, and guarantees. Each link opens a deeper SysGeeks guide.

What should you clarify first in a system design interview?

Answer. Identify the users and their core actions, then agree on scale, latency, availability, consistency and durability needs. Prioritize the requirements that change the architecture before drawing components.

Read more: Interview approach

How do you estimate capacity for a system design?

Answer. Estimate peak requests per second, data written per day, retention and concurrent work from the requirements. Use order-of-magnitude numbers to find likely bottlenecks, then size the parts that need different scaling strategies.

Read more: Capacity estimator

What is the difference between replication and sharding?

Answer. Replication keeps copies of the same data on multiple machines, which can improve read capacity and resilience. Sharding splits data across machines to spread storage and write load, but makes cross-shard operations harder.

Read more: Replication, Partitioning

When should you add a cache?

Answer. Cache data when repeated reads are costly and the hit rate makes the cache faster than recomputing or querying the source. Decide how stale a value may be and how writes invalidate it; caching adds consistency and failure cases.

Read more: Caching

When does a message queue help?

Answer. A queue decouples a producer from slower consumers and can absorb short bursts, giving workers time to process jobs asynchronously. It does not fix sustained overload: bound the queue, define retry and rejection behavior, and make handlers safe to retry.

Read more: Message queues

What does idempotency mean in system design?

Answer. An idempotent operation has the same effect when the same intent is repeated. Give each intent a stable key, claim it atomically, and reuse it across retries so a timeout does not trigger a second charge or job.

Read more: Idempotency

What is the difference between at-least-once and exactly-once delivery?

Answer. At-least-once delivery retries until acknowledgement, so a message may arrive more than once. Networks cannot guarantee exactly-once delivery; systems make repeated processing harmless with deduplication, idempotency, or conditional state changes.

Read more: Delivery guarantees

How do you stop a stale worker from overwriting a newer result?

Answer. Give each lease claim a monotonically increasing fencing token and require writes to present the current token. An expired worker may keep running, but storage rejects its stale token after another worker takes ownership.

Read more: Leases and fencing tokens

How can a database migration avoid downtime?

Answer. Expand the schema compatibly, backfill in bounded batches, capture concurrent writes, and compare old and new data before cutover. Keep the old path available until verification passes and rollback is no longer needed.

Read more: Online data migrations

What is backpressure, and when should a system apply it?

Answer. Backpressure limits incoming work when a downstream service cannot keep up. Bound queues and slow or reject producers so latency, memory use, and retry volume do not grow without limit.

Read more: Backpressure and capacity

What problem does the transactional outbox solve?

Answer. A database update can commit while publishing its event fails. Write the business change and an outbox record in one transaction, then relay the event with retries and make consumers handle duplicates.

Read more: Transactional outbox

How do you handle a hot partition?

Answer. A single high-volume key can overload one partition even when the rest of the cluster has capacity. Split or salt hot keys, or isolate hot tenants, then account for the extra work needed to merge reads and writes.

Read more: Partitioning

How should a payment system handle a timeout with an unknown outcome?

Answer. Treat the result as unknown rather than failed. Look up the provider operation by its stable reference and reconcile its state before retrying; never create a new charge just because the response timed out.

Read more: Reconciliation, Idempotency

What is load shedding?

Answer. Load shedding rejects or defers lower-priority work during overload to preserve capacity for critical requests. Pair it with bounded queues and clear retry behavior so rejected work does not create a retry storm.

Read more: Load shedding

When should work run asynchronously?

Answer. Use asynchronous processing when the caller does not need the result immediately and a durable queue can isolate slow work or smooth bursts. Return an operation status, then design retries, ordering, idempotency, and queue limits explicitly.

Read more: Asynchronous processing

Continue practising

Practice all 30 questions and use the interactive exercises at sysgeeks.com/investigations.